Digital VLSI System Design

Fall 2024

HW1

EE Department, Sharif University of Technology

Dr. Shabany

Deadline: 27 Mehr

---

1. Design an ALU with two 32-bit inputs, A and B, a 32-bit output X, a 1-bit output Z that becomes 1 if X is zero, and a 3-bit input Op that determines which of the following operations is performed on inputs A and B, with the result stored in X.

This ALU should be capable of performing the following operations:

- ADD
- SUB
- NOR
- OR
- NAND
- AND
- XNOR

If any of the above terms are unclear to you, search for more information on them.

Write the Verilog code for this ALU and verify its correctness using a testbench. You need to generate 100,000 random input data in MATLAB, store them in a file

called inputs.txt, and generate the correct output corresponding to those inputs in MATLAB, saving the results in a file.

The testbench should simply take the inputs from inputs.txt, pass them to the main Verilog module (ALU), and then compare the ALU's output with the MATLAB-generated output, reporting the accuracy percentage. Naturally, the accuracy should be 100%.

Thus, you should not perform any output calculations for the ALU in the testbench itself.

2. In this question, you will implement a sequence detector using state machines.

a) Assuming a single-bit input along with a valid signal that indicates the validity of the input data, and a single-bit output that indicates whether the sequence 10110 has been observed or not, implement a structure that performs this task using a Mealy state machine. Note that your code should consider overlapping; that is, if the input bit sequence is received as 101101101 in 9 consecutive clocks, the output bit sequence should be sent as 000010010.

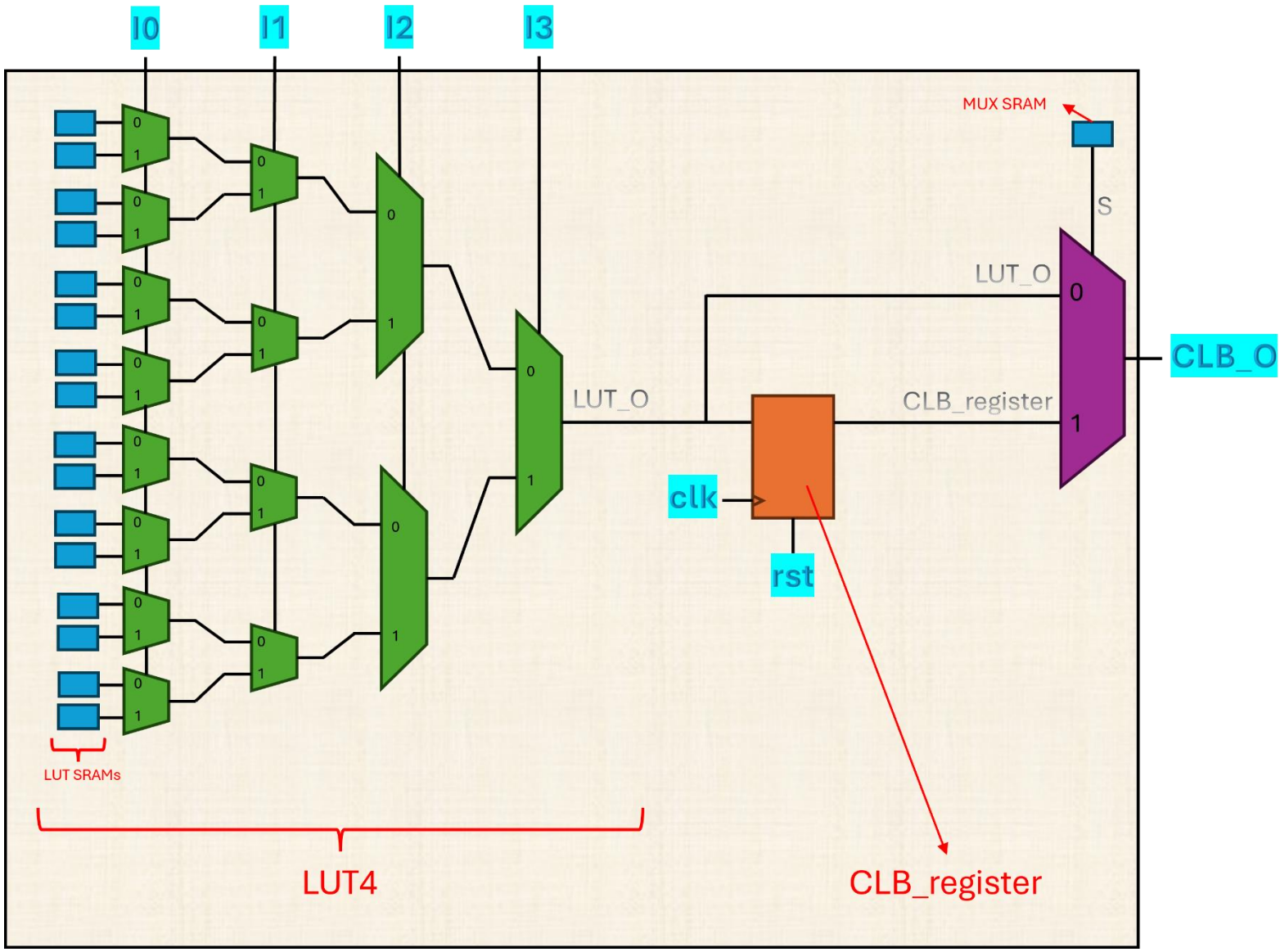
b) By writing a testbench for this part, verify and validate the functionality of your circuit. You must generate a random input bit sequence that includes at least 10000 bits in bitstream and give it to the main module (sequence detector).

3. In this section, you will create a very simple FPGA and test your design via some combinational and some sequential circuits. You must use MATLAB or Python to verify your design; by creating the exact same program as it is in your HDL.

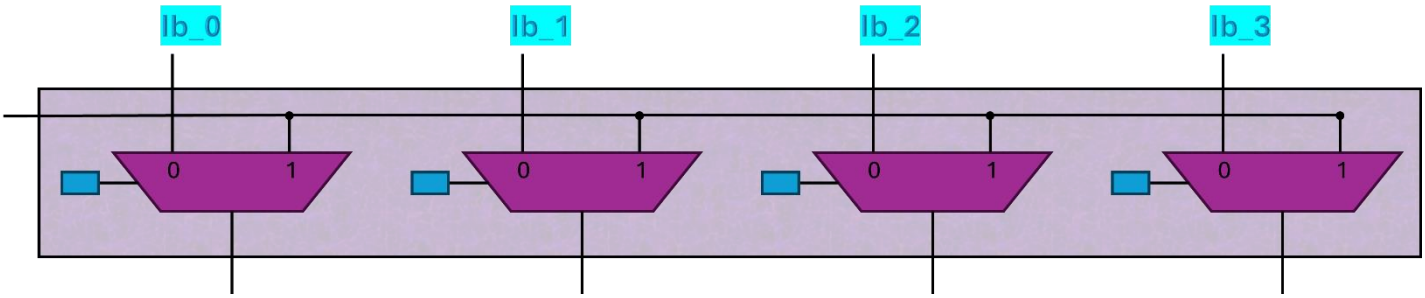
As you can see, a CLB (schematic 1) contains a LUT4 for the combinational section and a flip flop. According to wire “S”, the output of the LUT can either be flopped or not in order to be outputted from the CLB. Each CLB has 17 SRAM cells, 16 for the LUT4 and 1 for the selector of the mux, “S”. Your design must contain 2 CLBs named CLBa and CLBb and 1 connector (schematic 3).

The connector (schematic 2) is to create a versatile connection between the 2 CLBs. It connects the output of CLBa to all the 4 inputs of CLBb via 4 muxes. Each of the muxes has a selector that chooses the input of the CLBb, either it is the output of the CLBa or the input of the FPGA. The selectors within the connector are SRAMs programmed in advance to make sure our desired connections are created.

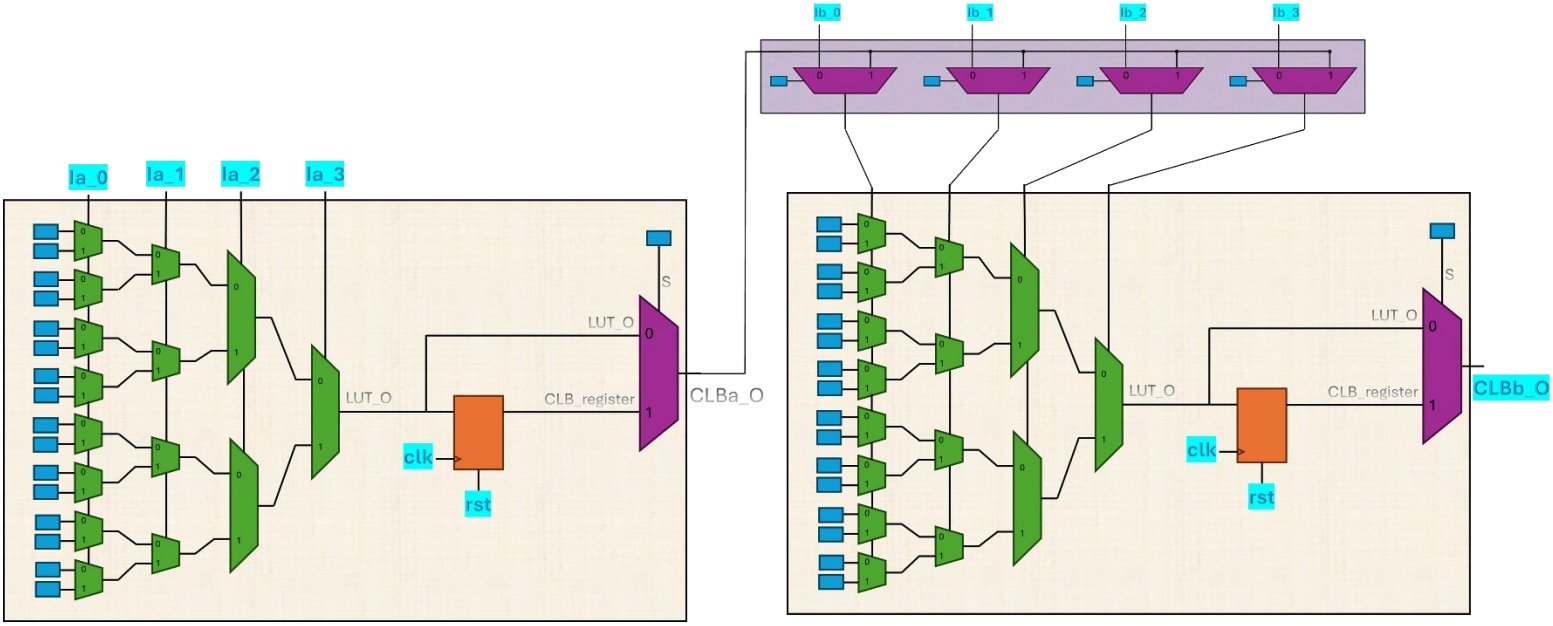
You must have 38 SRAMs in the whole design. You need to create a 38-bit memory for the SRAMs in your design and put the bitstream in the SRAM via your testbench. You may synthesize your design manually to fit your small FPGA and enter the created bitstream into the SRAMs via testbench. Your FPGA module must have 10 inputs, 4 for each of the 2 CLBs in addition to a “clk” and a “rst” for the CLB registers. The FPGA module has only 1 output named “CLBb_O”.



Schematic 1, the “CLB” module



Schematic 2, the “connector” module



Schematic 3, the complete “FPGA” module

You need to implement the following functions using your FPGA. The complete HDL of these functions, as well as the synthesized netlist HDL and the bitstream, must be present in your report.

1) The following truth table (combinational):

a	b	c	d	f
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

2) f = (a&b)|(!c|d) (combinational)
3) f = ((a&b)|(!c|d))^e (output must be registered)
4) f = b>a (a and b are unsigned and 3 bits each and combinational)
5) f = b>a (a and b are unsigned and 3 bits each and pipelined)